LEXICON

NOSQL & MONGODB

# Asking better questions

Query operators, arrays, sorting, and changing documents

LEXICON

# Today

- After today you can:
  - Filter with operators like $gt, $in and $or
  - Search inside objects (dot notation) and arrays
  - Choose fields with projection, and sort, limit and count
  - Change documents with update_one, update_many and update operators
  - Delete documents safely

LEXICON

# Recap

- MongoDB stores documents in collections
- A document is like a Python dictionary, with lists and objects inside
- Every document has a unique _id
- pymongo = talk to MongoDB from Python
- Filters are dictionaries: {"city": "Uppsala"}

LEXICON

# Quick questions

1. What is a collection, and what is it called in SQL?
2. What does db.customers.find() give you, and how do you see the documents?
3. Why does a misspelled field name give 0 results instead of an error?

LEXICON

# Comparison operators

| MongoDB | SQL | Means |
|---|---|---|
| $eq | = | equal |
| $ne | <> | not equal |
| $gt / $gte | > / >= | greater than (or equal) |
| $lt / $lte | < / <= | less than (or equal) |
| $in / $nin | IN / NOT IN | in the list / not in the list |

```
{"price": {"$lt": 500}}                    # price < 500
{"category": {"$in": ["Shoes", "Clothing"]}}
```

LEXICON

# AND and OR

- Same field twice with AND? Put both operators together: {"price": {"$gt": 200, "$lt": 600}}

```
# AND: put several fields in the same filter
{"category": "Shoes", "price": {"$gt": 1000}}

# OR: a list of filters
{"$or": [{"category": "Shoes"}, {"stock": 0}]}
```

LEXICON

# Inside objects: dot notation

- Use a dot to reach a field inside an object
- The whole name needs quotes: "details.color"
- Not the same as {"details": {"color": "black"}} (exact match of the whole object!)

```
{"details.color": "black"}      # products whose details.color is black
```

LEXICON

# Searching in arrays

```
{"tags": "winter"}                          # tags contains "winter"
{"tags": {"$all": ["winter", "cotton"]}}    # contains both
{"tags": {"$size": 2}}                      # exactly 2 tags
{"sizes": 44}                               # works with numbers too
```

LEXICON

# Does the field exist?

- Missing field and null are not the same thing
- But {"city": None} matches both

```
{"phone": {"$exists": True}}     # customers that have a phone field
{"city": {"$exists": False}}     # customers without a city field
```

LEXICON

# Projection: choose your fields

- Second argument to find()
- 1 = include, 0 = leave out
- _id is included unless you say "_id": 0

```
db.products.find({}, {"name": 1, "price": 1, "_id": 0})
# SELECT name, price FROM products
```

LEXICON

# Sort, limit and skip

- 1 = ascending (smallest first), -1 = descending

```
db.products.find().sort("price", -1).limit(3)   # 3 most expensive
db.products.find().sort("name", 1)               # A to Z
db.products.find().sort("price", 1).skip(5).limit(5)
```

LEXICON

# Counting and distinct values

```
db.products.count_documents({"stock": 0})   # 2
db.products.distinct("category")            # ["Clothing", "Accessories", "Shoes"]
```

LEXICON

# Changing documents

- First: which documents (filter). Then: what to change (update operator)
- result.matched_count and result.modified_count tell you what happened

```
db.products.update_one({"_id": 10}, {"$set": {"price": 199}})
db.products.update_many({"category": "Shoes"}, {"$set": {"on_sale": True}})
```

LEXICON

# Update operators

| Operator | Does | Example |
|---|---|---|
| $set | Set a value (adds the field if missing) | {"$set": {"price": 199}} |
| $unset | Remove a field | {"$unset": {"phone": ""}} |
| $inc | Add to a number | {"$inc": {"stock": -1}} |
| $push | Add to an array | {"$push": {"tags": "sale"}} |
| $addToSet | Add to an array, if not already there | {"$addToSet": {"tags": "sale"}} |
| $pull | Remove from an array | {"$pull": {"tags": "sale"}} |

LEXICON

# No Revert button!

- Run find() or count_documents() with the same filter first

```
db.products.update_many({}, {"$set": {"price": 0}})   # every product is free
db.orders.delete_many({})                               # every order is gone
```

**Empty filter = EVERY document. And there is no undo.**

LEXICON

# Upsert: update or insert

- Found? It's updated
- Not found? It's inserted

```
db.customers.update_one(
    {"email": "nora@example.com"},
    {"$set": {"first_name": "Nora", "city": "Lund"}},
    upsert=True)
```

LEXICON

# Deleting

```
db.orders.delete_one({"_id": 7})
result = db.orders.delete_many({"status": "cancelled"})
print(result.deleted_count)
```

LEXICON

# Remember

- Operators: $gt, $lt, $in, $ne... written as {"field": {"$op": value}}
- "details.color" reaches inside objects. {"tags": "winter"} searches in arrays
- find(filter, projection).sort(...).limit(...)
- update_one / update_many with $set, $inc, $push, $pull, $addToSet
- Empty filter = everything. No undo: check with find first!

LEXICON

# This afternoon: lab

**Open the exercise PDF. Work at your own pace, and ask for help when you get stuck.**

LEXICON

# Coming up tomorrow

**Tomorrow: how to design good documents, and the aggregation pipeline (MongoDB's GROUP BY).**